

Lab 02 — Secure VPC & Data Protection

Practise public, private and isolated network separation, troubleshoot App → DB connectivity, and verify KMS-encrypted S3 storage.

Lab environment: Docker Desktop + LocalStack — runs locally, no AWS account needed

Estimated time: 35–40 minutes

SAA-C03 domain: Design Secure Architectures

Overview

Local component	AWS equivalent
Published port 8080	Internet gateway / public subnet
lab2-alb (NGINX reverse proxy)	Application Load Balancer
private-net (internal Docker network)	Private subnet with security-group isolation
lab2-app	Private application tier
isolated-net (internal Docker network)	Isolated database subnet / NACL separation
lab2-db (MariaDB)	Database tier (Amazon RDS)
LocalStack KMS + S3	AWS KMS key + SSE-KMS encrypted Amazon S3 bucket

Docker networks are a teaching approximation. They do not reproduce VPC route tables, security groups, NACLs, NAT gateways, ACM or VPC endpoints exactly.

Step 1 — Get the lab source and start the lab

The repo lives in `~/Downloads/aws-saa`. The paths are absolute, so these commands work from any folder. `git clone` is needed only once: if the repo is already there it prints `already exists`, and the next line still takes you into this lab.

```
cd ~/Downloads
git clone https://github.com/mohanpothula/aws-saa.git
cd ~/Downloads/aws-saa/labs/lab-02-secure-vpc-data-protection
docker compose up -d
until docker compose logs localstack 2>&1 | grep -q "created successfully"; do sleep 2; done;
echo READY
docker compose ps
```

The app and database are **deliberately** on separate internal networks — finding and fixing that is Step 6.

Step 2 — Test the public entry point

Task: Reach the application through the simulated ALB.

```
curl http://localhost:8080
```

Expected result: Response contains `LAB 2 - Private Application Tier`.

Step 3 — Validate the private app

Task: Prove the app has no direct host port.

```
docker port lab2-app
docker inspect lab2-app --format '{{json .NetworkSettings.Networks}}'
```

Expected result: `docker port` prints nothing — `lab2-app` has no host-published port.

Step 4 — Validate the isolated database

Task: Prove MariaDB is not directly exposed.

```
docker port lab2-db
docker inspect lab2-db --format '{{json .NetworkSettings.Networks}}'
```

Expected result: `docker port` prints nothing — `lab2-db` has no host-published port.

Step 5 — Network security challenge

Task: Compare the ALB, App and DB networks and identify why App cannot initially reach DB.

```
docker inspect lab2-alb --format '{{json .NetworkSettings.Networks}}'
docker inspect lab2-app --format '{{json .NetworkSettings.Networks}}'
docker inspect lab2-db --format '{{json .NetworkSettings.Networks}}'
```

Expected result: App is on `private-net` only; DB is on `isolated-net` only. They share no network, so there is no path between them.

Step 6 — Fix App → DB connectivity

Task: Edit `docker-compose.yml` so `app` also joins `isolated-net`. Do **not** publish DB port 3306.

First prove the path is broken: `docker exec lab2-app nc -zv db 3306` prints `nc: bad address 'db'`, because the app cannot even see the database. Then open `docker-compose.yml` in any editor, add `isolated-net` to the `app` service's `networks:` list, and save the file. Do not change the `db` service, and do not give it a `ports:` section:

```
app:
  image: nginx:alpine
  container_name: lab2-app
  volumes:
    - ./app-index.html:/usr/share/nginx/html/index.html:ro
  networks:
    - private-net
    - isolated-net      # add this line
```

Check that the file saved: `grep -A8 '^ app:' docker-compose.yml` must show `isolated-net`. Then recreate the app and test the path:

```
docker compose up -d --force-recreate app alb
docker exec lab2-app nc -zv db 3306
docker port lab2-db
```

Expected result: `db (...:3306) open` — the database is reachable from the app, yet `docker port lab2-db` still prints nothing.

Why not `apk add` first? `isolated-net` is an **internal** network with no route to the internet, so package installs fail inside it — which is exactly the isolation you are testing. `nc` is already built into the image's BusyBox. Recreate only `app` and `alb`: recreating every service would also restart LocalStack, which keeps its data in memory, so its bucket would vanish until the init script re-ran and Step 7 could fail with `NoSuchBucket`. `alb` is recreated alongside `app` because NGINX looks up the app's address only when it starts.

Step 7 — Verify KMS / S3 encryption

Task: Confirm the bucket is encrypted with a KMS key.

```
docker exec aws-saa-lab2 awslocal kms list-aliases
docker exec aws-saa-lab2 awslocal s3api get-bucket-encryption --bucket lab2-secure-data
docker exec aws-saa-lab2 awslocal s3 ls s3://lab2-secure-data
```

Expected result: Three separate results, one per command: (1) the key alias `alias/lab2-s3-key`; (2) `SSEAlgorithm` is `aws:kms`, with the same key ID as in (1); (3) the file `secure.txt` in the bucket.

Each command runs `awslocal` inside the LocalStack container and returns to your terminal, so you can paste all three at once. If `get-bucket-encryption` reports `NoSuchBucket`, LocalStack was restarted and its in-memory data was reset: wait for `READY` again (`until docker compose logs localstack 2>&1 | grep -q "created successfully"; do sleep 2; done; echo READY`) and re-run the commands.

Verification

Check	Expected
Step 2 — Test the public entry point	Response contains <code>LAB 2 - Private Application Tier</code>
Step 3 — Validate the private app	<code>docker port</code> prints nothing — <code>lab2-app</code> has no host-published port
Step 4 — Validate the isolated database	<code>docker port</code> prints nothing — <code>lab2-db</code> has no host-published port
Step 5 — Network security challenge	App is on <code>private-net</code> only; DB is on <code>isolated-net</code> only. They share no network, so there is no path between them
Step 6 — Fix App → DB connectivity	<code>db (...:3306) open</code> — the database is reachable from the app, yet <code>docker port lab2-db</code> still prints nothing
Step 7 — Verify KMS / S3 encryption	Three separate results, one per command: (1) the key alias <code>alias/lab2-s3-key</code> ; (2) <code>SSEAlgorithm</code> is <code>aws:kms</code> , with the same key ID as in (1); (3) the file <code>secure.txt</code> in the bucket

Completion check

- ☐ Public endpoint works through the ALB
- ☐ App and DB are not directly exposed
- ☐ You repaired the App → DB network path
- ☐ S3 encryption reports `aws:kms`

Cleanup

```
docker compose down -v
git checkout -- docker-compose.yml # undo the Step 6 edit so the lab can be repeated
```

`-v` also removes the lab's volumes, so the next run starts from a clean state.

Reference

- Lab source code:
<https://github.com/mohanpothula/aws-saa/tree/main/labs/lab-02-secure-vpc-data-protection>
- Original student guide: `courseware/AWS_SAA_Lab2_Student_Guide.docx`
- [What is Amazon VPC?](#)
- [VPC security groups](#)
- [VPC network ACLs](#)
- [What is an Application Load Balancer?](#)
- [AWS KMS concepts](#)

- [S3 server-side encryption with AWS KMS \(SSE-KMS\)](#)
- [LocalStack documentation](#)
- [Compose file reference — networks](#)
- [NGINX proxy module \(proxy_pass, proxy_cache\)](#)